

크로스 플랫폼 프로그래밍

3주: 스타일링 기초

강의: 정종수



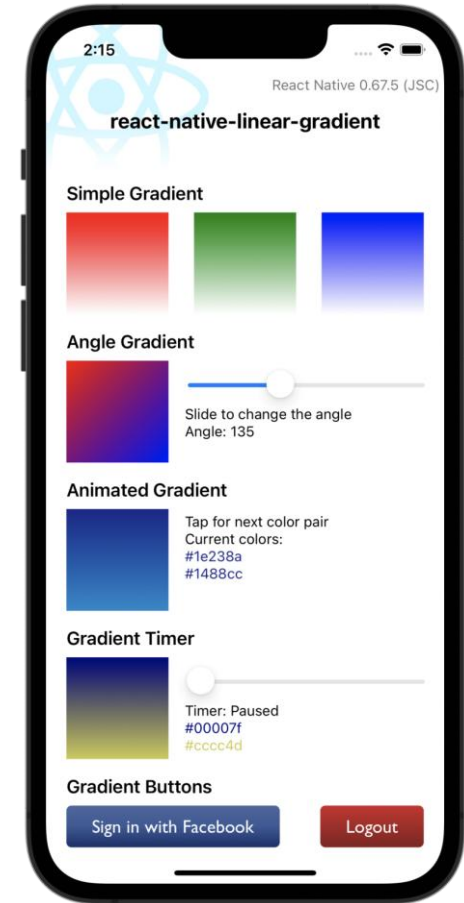
강의 내용



1. 정렬
2. 스타일링
3. 인라인 스타일링
4. 클래스 스타일링
5. 여러 개의 스타일 적용
6. 외부 스타일 이용하기
7. Flex와 범위
8. 실습

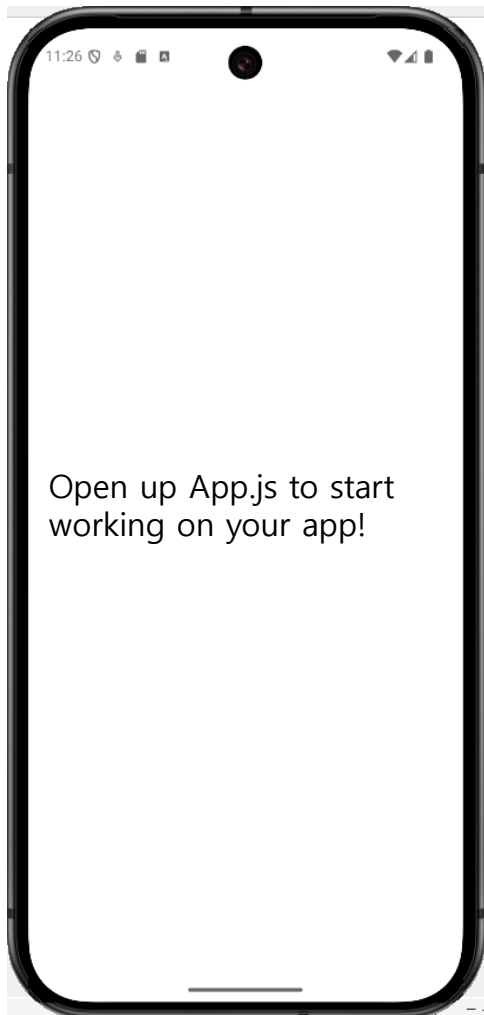
스타일링이란 ?

- 화면요소(텍스트, 버튼, 이미지 등)의 색상, 크기, 간격, 배치를 조정해 시각적으로 보기 좋게 꾸미는 작업.
- 리액트 네이티브에서의 스타일링은 웹 프로그래밍에서 사용하는 CSS와 약간 차이가 있음.
- 컴포넌트 속성에 인라인 스타일을 적용하는 방법과 스타일 시트(StyleSheet)에 정의된 스타일을 사용하는 방법이 있음.



1. 정렬

- 처음 시작하는 JSX: expo init “프로젝트 명 ”, 실행: npm start



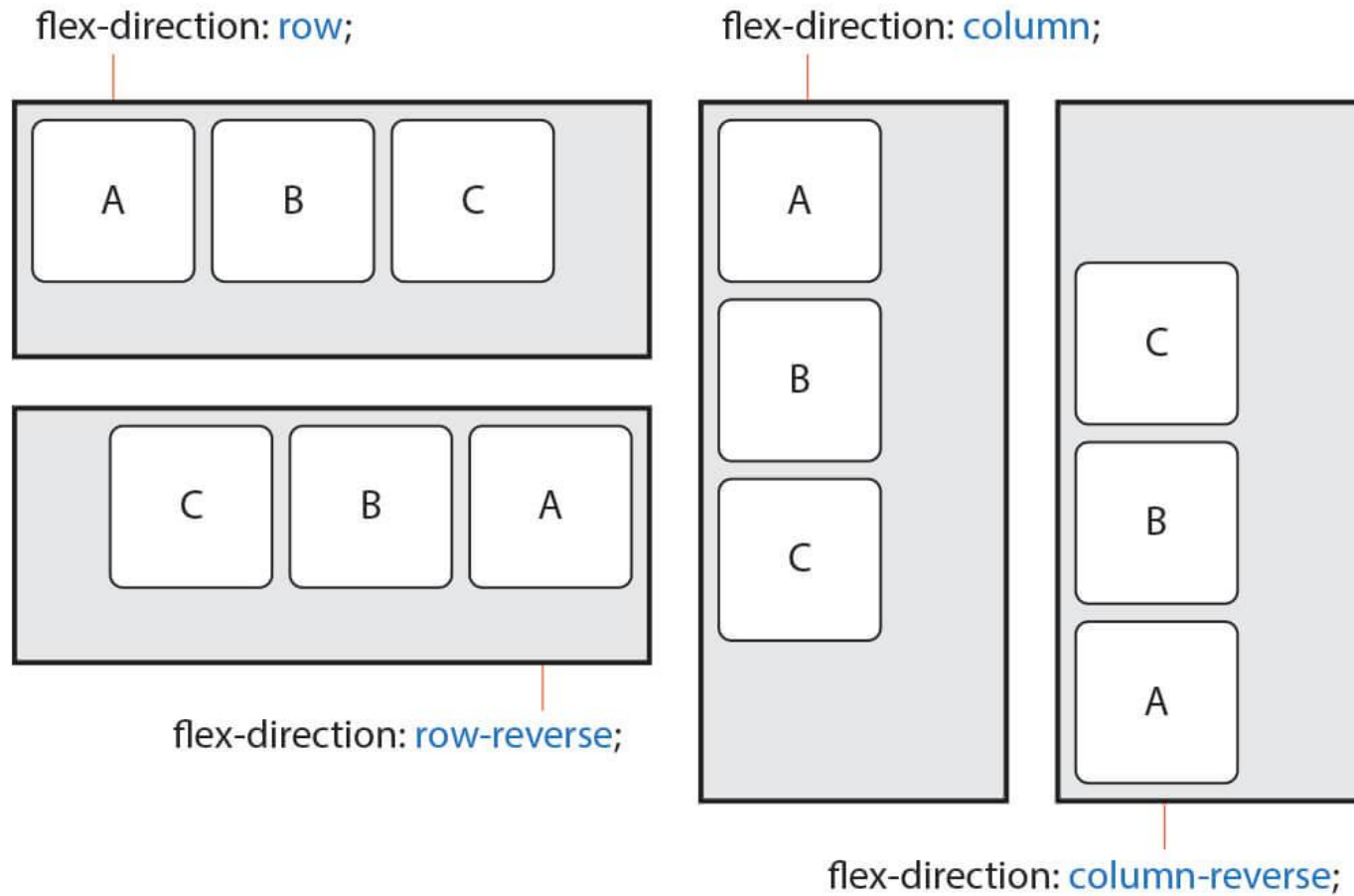
```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
```

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

1. 정렬

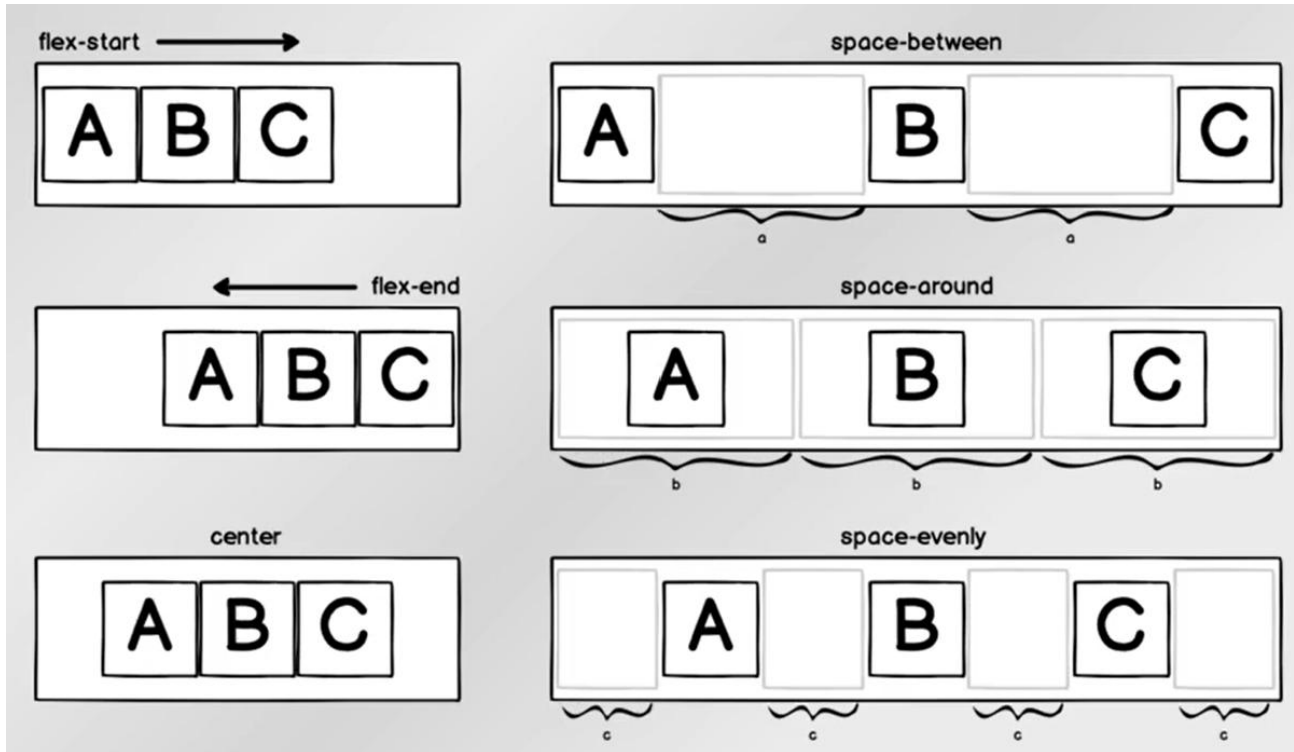
컴포넌트를 정렬하는 방법: flexDirection



- Column: 세로방향으로 정렬(기본값)
- Column-Reverse: 세로방향 역순 정렬
- Row: 가로방향으로 정렬
- Row-Reverse: 가로방향 역순 정렬

1. 정렬

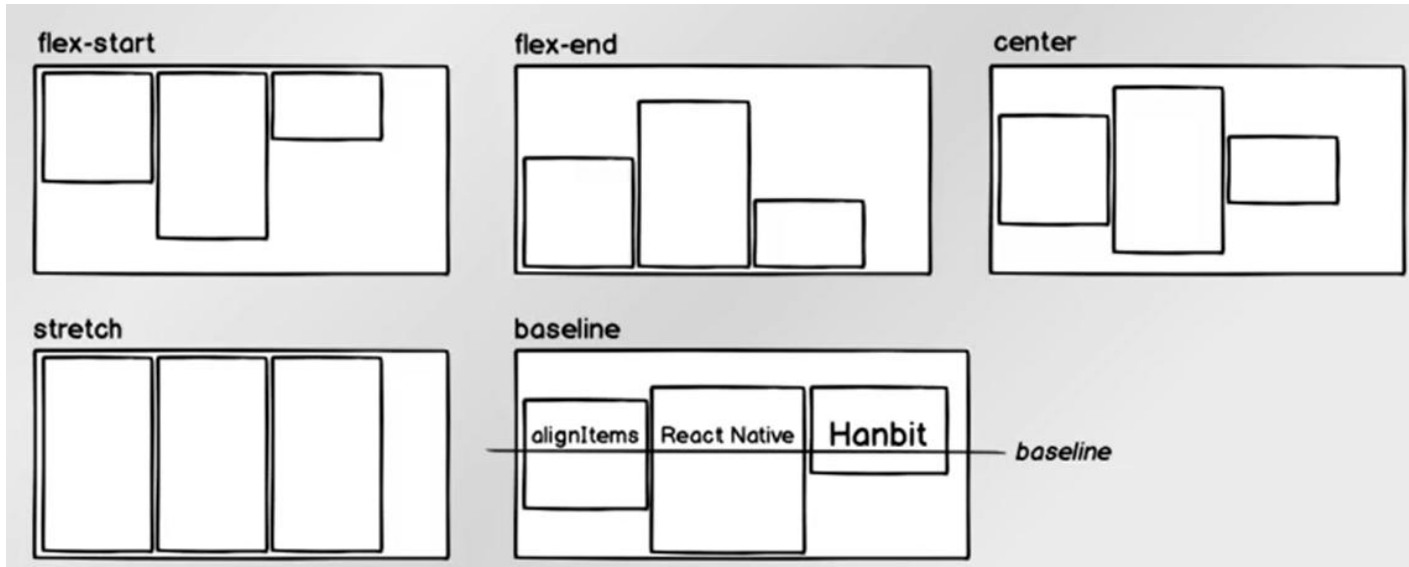
■ 컴포넌트를 정렬하는 방법: justifyContent



- `flex-start`: 시작점에서부터 정렬
- `flex-end`: 끝에서부터 정렬
- `center`: 중앙 정렬
- `space-between`: 컴포넌트 사이의 공간을 동일하게 만들어 정렬
- `space-around`: 컴포넌트 각각의 주변 공간을 동일하게 만들어 정렬
- `space-evenly`: 컴포넌트 사이와 양 끝에 동일한 공간을 만들어서 정렬

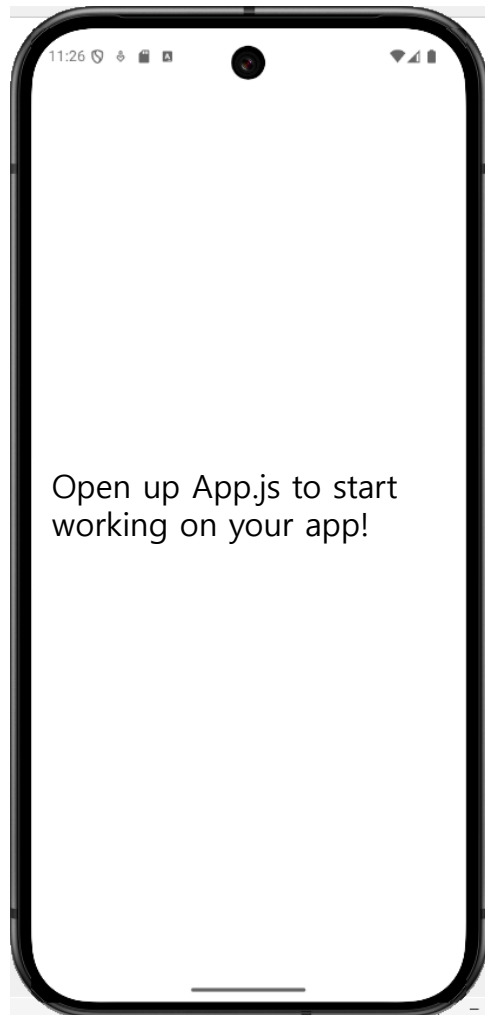
1. 정렬

▪ 컴포넌트를 정렬하는 방법: `alignItems`



- `flex-start`: 시작점에서부터 정렬
- `flex-end`: 끝에서부터 정렬
- `center`: 중앙 정렬
- `stretch`: `alignItems`의 방향으로 컴포넌트 확장
- `Baseline`: 컴포넌트 내부의 텍스트(text) 베이스라인(baseline)을 기준으로 정렬

2. 스타일링: 영어

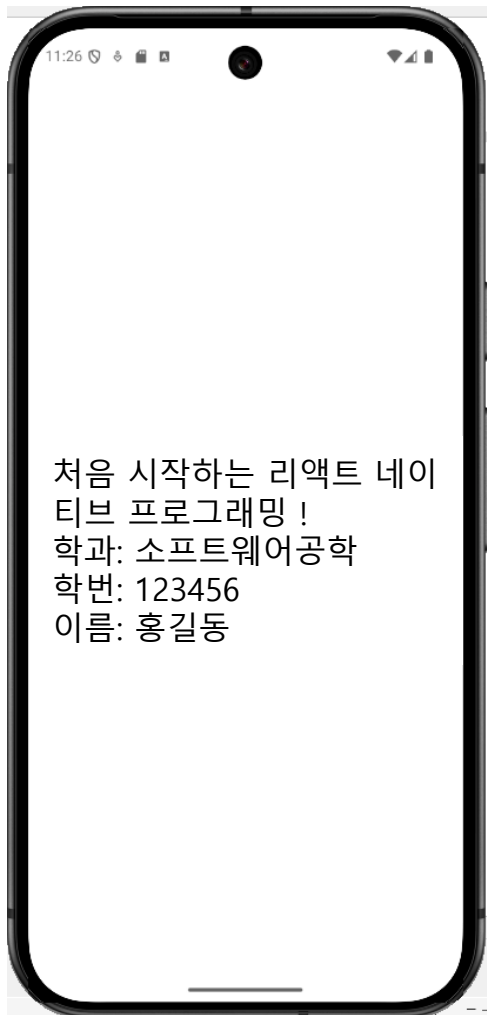


```
import { StatusBar } from 'expo-status-bar';  
import { StyleSheet, Text, View } from 'react-native';
```

```
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text>Open up App.js to start working on your app!</Text>  
      <Text>First My Project !</Text>  
      <StatusBar style="auto" />  
    </View>  
  );  
}
```

```
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    backgroundColor: '#fff',  
    alignItems: 'center',  
    justifyContent: 'center',  
  },  
});
```

2. 스타일링: 한글



```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
```

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>처음 시작하는 리액트 네이티브 프로그래밍 !{"\n"} 학과:
      소프트웨어공학 {"\n"}학번: 123456{"\n"}이름: 홍길동 </Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

2. 스타일링: 주석

- 리액트 네이티브에서의 주석 처리는 자바스크립트에서의 주석과 약간 차이가 있음.



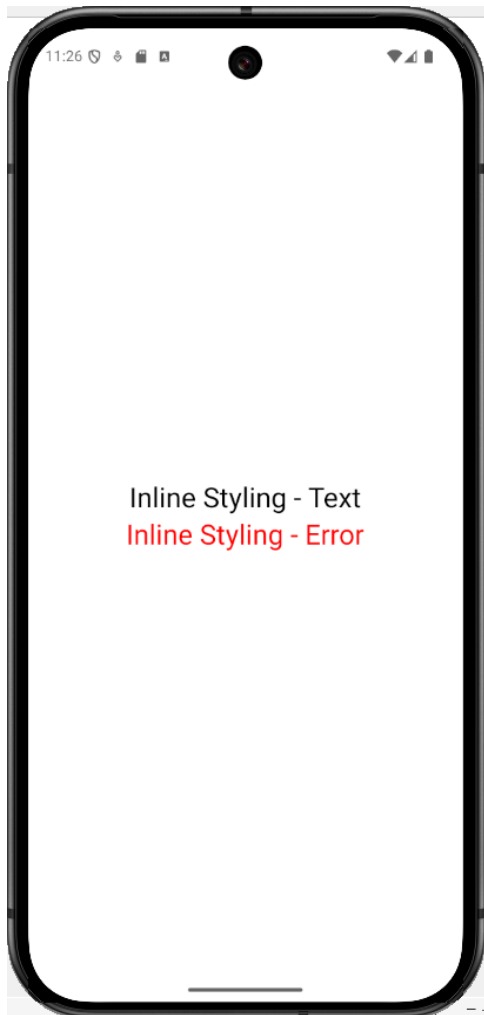
```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>처음 시작하는 리액트 네이티브 프로그래밍 !</Text>
      { /* <Text>처음 시작하는 리액트 네이티브 프로그래밍 !</Text> */ }
      <StatusBar style="auto" />
    </View>
  );
}

// 과목명: 리액트 네이티브 프로그래밍
/* 이름: 홍길동 */
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

3. 인라인 스타일링

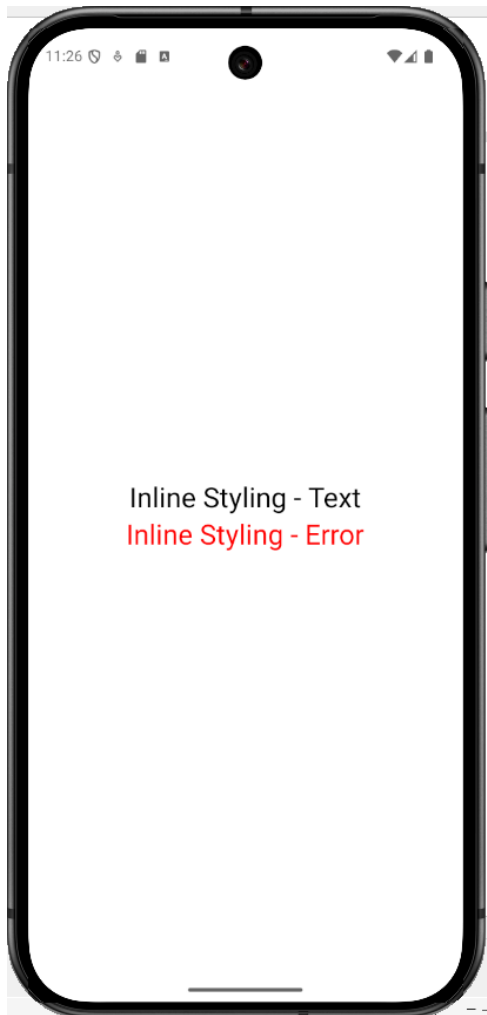
- HTML의 인라인 스타일링 처럼 컴포넌트에 직접 스타일을 입력하는 방식



```
export default function App() {
  return (
    <View
      style={{
        flex: 1,
        backgroundColor: "#fff",
        alignItems: "center",
        justifyContent: "center",
      }}>
      <Text
        style={{
          padding: 10,
          fontSize: 26,
          fontWeight: "600", // 숫자가 아닌 문자열
          color: "black",
        }}>
        Inline Styling - Text
      </Text>
      <Text
        style={{
          padding: 10,
          fontSize: 26,
          fontWeight: "400", // 숫자가 아닌 문자열
          color: "red",
        }}>
        Inline Styling - Error
      </Text>
    </View>
  );
}
```

4. 클래스 스타일링

- 컴포넌트의 태그에 직접 입력하는 방식이 아니라 스타일시트에 정의된 스타일을 사용하는 방법

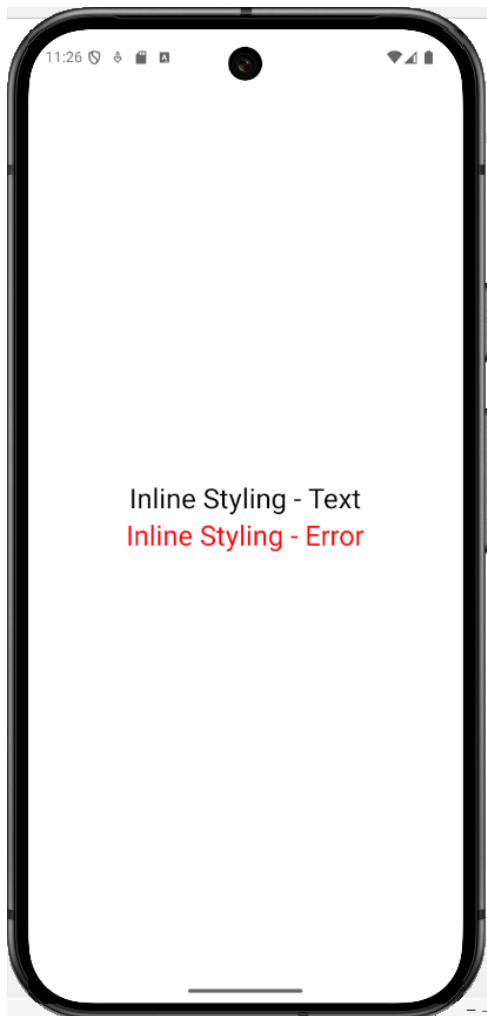


```
export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Inline Styling - Text</Text>
      <Text style={styles.error}>Inline Styling - Error</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  text: {
    padding: 10,
    fontSize: 26,
    fontWeight: '600',
    color: 'black',
  },
  error: {
    padding: 10,
    fontSize: 26,
    fontWeight: '400',
    color: 'red',
  },
});
```

5. 여러 개의 스타일 적용

- 여러 개의 스타일을 적용해야 할 경우는 배열을 이용하여 Style 속성에 여러 개의 스타일을 적용.



```
export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Inline Styling - Text</Text>
      <Text style={[styles.text, styles.error]}>Inline Styling - Error</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  text: {
    padding: 10,
    fontSize: 26,
    fontWeight: '600',
    color: 'black',
  },
  error: {
    fontWeight: '400',
    color: 'red',
  },
});
```

6. 외부 스타일 이용하기

- 외부 파일에 스타일을 작성하고 컴포넌트에서 외부 파일에 정의된 스타일을 이용하기.



- App.js 파일

```
import React from "react";
import { View, Text } from "react-native";
import { viewStyles, textStyles } from "../src/styles";

export default function App() {
  return (
    <View style={viewStyles.container}>
      <Text style={textStyles.text}>Inline Styling - Text</Text>
      <Text style={textStyles.error}>Inline Styling - Error</Text>
    </View>
  );
}
```

6. 외부 스타일 이용하기

- 외부 파일에 스타일을 작성하고 컴포넌트에서 외부 파일에 정의된 스타일을 이용하기.



- src 폴더생성 → styles.js 파일 만들기

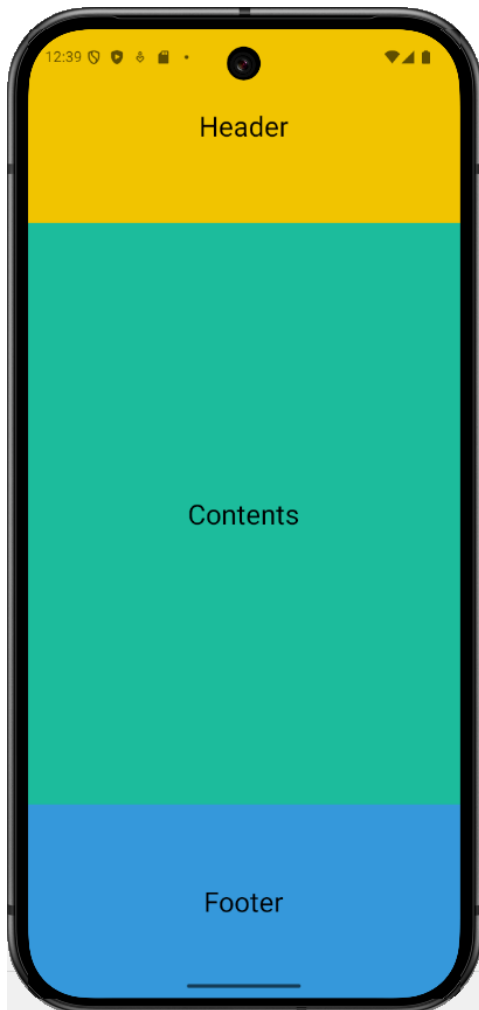
```
import { StyleSheet } from "react-native";

export const viewStyles = StyleSheet.create({
  container:{
    flex: 1,
    backgroundColor: '#fff',
    alignItems:'center',
    justifyContent: 'center',
  }
});

export const textStyles = StyleSheet.create({
  text:{
    padding: 10,
    fontSize: 26,
    fontWeight: '600',
    color:'black',
  },
  error:{
    fontWeight:'400',
    color:'red',
  },
});
```

7. flex와 범위

- 리액트 네이티브에서 Flex는 화면 레이아웃을 비율로 나누는 속성임.



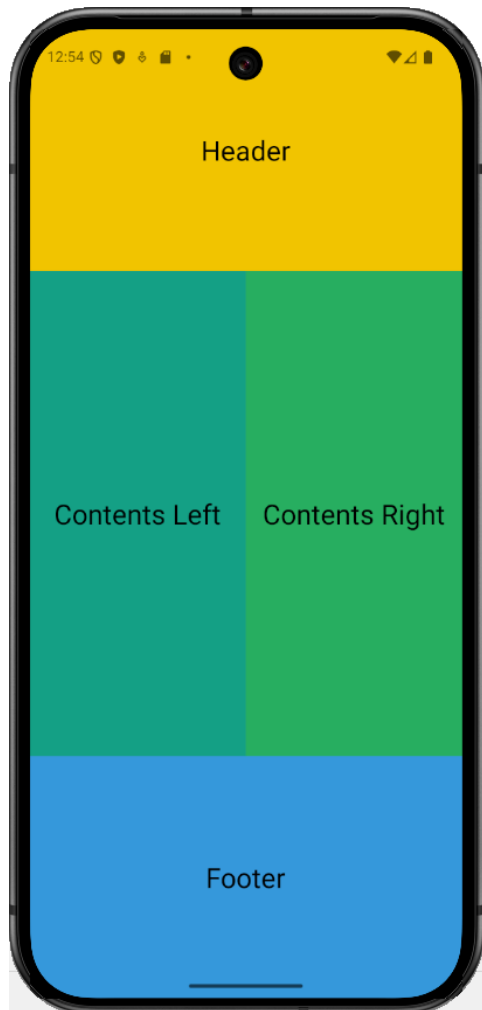
```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <Text
          style={styles.text}>Header</Text>
      </View>
      <View style={styles.contents}>
        <Text
          style={styles.text}>Contents</Text>
      </View>
      <View style={styles.footer}>
        <Text
          style={styles.text}>Footer</Text>
      </View>
    </View>
  );
};
```

```
const styles = StyleSheet.create({
  container: {
    flex:1,
  },
  header: {
    flex:1,
    backgroundColor: '#f1c40f',
    alignItems: "center",
    justifyContent: "center",
  },
  contents: {
    flex: 3,
    backgroundColor: '#1abc9c',
    alignItems: "center",
    justifyContent: "center",
  },
  footer: {
    flex:1,
    backgroundColor: '#3498db',
    alignItems: "center",
    justifyContent: "center",
  },
  text: {
    fontSize: 26,
  },
});
```

7. flex와 범위

■ 화면 레이아웃 예제.



```
{/* Contents */} //컨텐츠 부부에 적용
<View style={styles.contents}>
  <View style={styles.contentsLeft}>
    <Text style={styles.text}>Contents Left</Text>
  </View>
  <View style={styles.contentsRight}>
    <Text style={styles.text}>Contents Right</Text>
  </View>
</View>
```

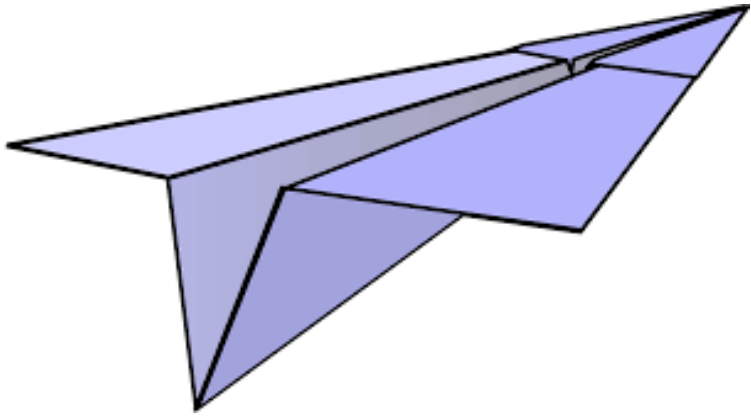
```
contents: {
  flex: 2,
  flexDirection: "row", // 가로 분할
},
contentsLeft: {
  flex: 1,
  backgroundColor: "#16a085",
  alignItems: "center",
  justifyContent: "center",
},
contentsRight: {
  flex: 1,
  backgroundColor: "#27ae60",
  alignItems: "center",
  justifyContent: "center",
},
```

8. 실습



- 이런닝 캠퍼스 제출 목록

Emulator 화면 캡처



Q&A

E-mail: jungjongsoo@yahoo.com